Multilevel Inheritance: Multilevel inheritance in Python occurs when a class inherits from a another drived class, creating a chain of inheritance similar to a family tree (Grandparent → Parent → Child).

In this structure, the "grandchild" class has access to all methods and attributes from both its immediate parent and its grandparent (as shown in Figure 3.5).

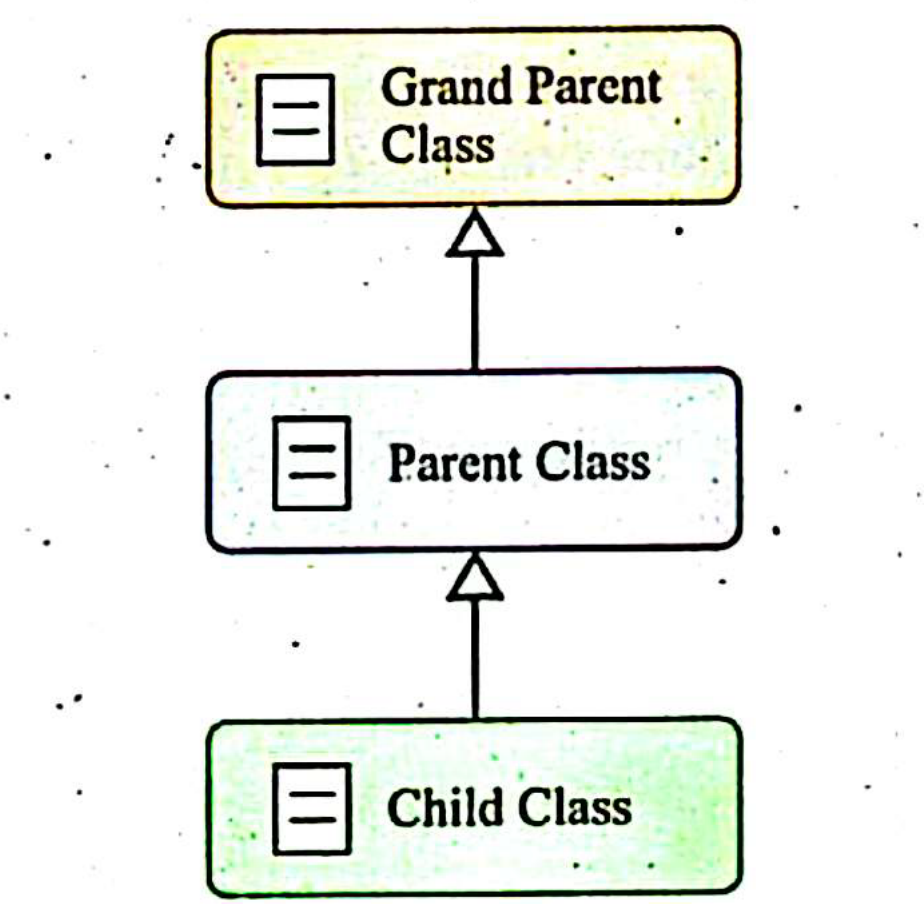


Figure 3.5: Multilevel Inheritance

Example:

In this example, ElectricCar inherits from Car, which in turn inherits from Vehicle.

```
class Vehicle:          # Grandparent Class
    def start_engine(self):
        print("Engine started.")

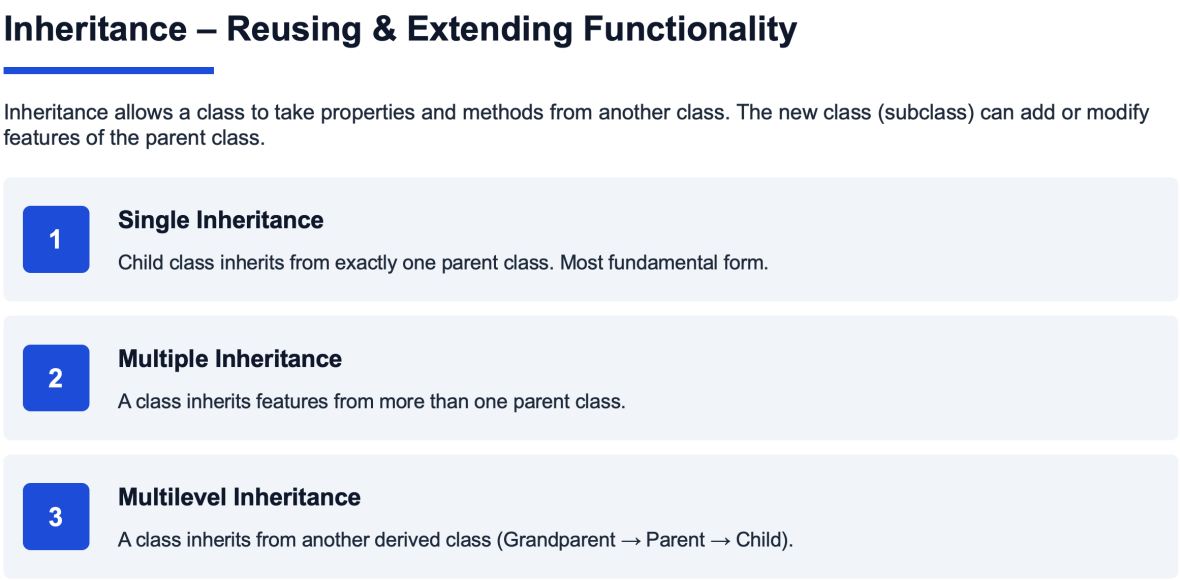
class Car(Vehicle):                 # Parent Class
    def drive(self):
```

```
        print("Car is driving.")

class ElectricCar(Car):                 # Child Class
(Grandchild to Vehicle)
    def charge(self):
        print("Battery is charging.")

my_tesla = ElectricCar()
my_tesla.start_engine()     # Inherited from
Vehicle
my_tesla.drive()            # Inherited from Car
my_tesla.charge()           # Defined in
ElectricCar
```

Method Overriding: A child class can provide a specific implementation of a method that is already provided by its parent. You can use the `super()` function to call the parent's version of that method.



📝 Short Questions

1. Define Multilevel inheritance.
2. Demonstrate Multilevel inheritance through diagram.
3. Distinguish between multiple inheritance and multilevel inheritance.
4. **What three-tier chain illustrates multilevel inheritance in the textbook's code?**
    1. **Answer:** Vehicle (Grandparent) → Car (Parent) → ElectricCar (Child).

🖋️ Long Questions

1. Describe the concept of Multilevel inheritance. Illustrate your answer with diagrams. Write a python program that show multiple inheritance.
2. Describe the concept of inheritance and analyze its key variations (Single, Multiple, and Multilevel). Illustrate your answer with diagrams showing how structural inheritance lines connect.
3. **Compare multiple inheritance and multilevel inheritance, explaining their structural differences.**

Class Activity:

Create a Vehicle class and extend it with Car and Bike subclasses. Add unique methods to each subclass and demonstrate inheritance.

Class Activity:

Create a Vehicle class and extend it with Car and Bike subclasses. Add unique methods to each subclass and demonstrate inheritance.

```
# Base Class (Parent Class)
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def start(self):
        print(f"{self.brand} {self.model} is starting...")

    def stop(self):
        print(f"{self.brand} {self.model} has stopped.")

    def display_info(self):
        print(f"Brand: {self.brand}, Model: {self.model}")


# Subclass 1: Car (inherits from Vehicle)
class Car(Vehicle):
    def __init__(self, brand, model, doors):
        super().__init__(brand, model)   # Call parent constructor
        self.doors = doors

    # Unique method for Car
```

```
    def open_trunk(self):
        print(f"{self.brand} {self.model}'s trunk is now open.")

    def display_info(self):   # Method Overriding
        super().display_info()
        print(f"Number of Doors: {self.doors}")


# Subclass 2: Bike (inherits from Vehicle)
class Bike(Vehicle):
    def __init__(self, brand, model, engine_cc):
        super().__init__(brand, model)
        self.engine_cc = engine_cc

    # Unique method for Bike
    def wheelie(self):
        print(f"{self.brand} {self.model} is doing a wheelie!")

    def display_info(self):   # Method Overriding
        super().display_info()
        print(f"Engine Capacity: {self.engine_cc} cc")


# ========== Demonstration ==========
print("----- Car Object -----")
my_car = Car("Toyota", "Corolla", 4)
my_car.start()          # Inherited method
```

```
my_car.open_trunk()      # Unique method of Car
my_car.display_info()    # Overridden method
my_car.stop()            # Inherited method

print("\n----- Bike Object -----")
my_bike = Bike("Honda", "CG-125", 125)
my_bike.start()          # Inherited method
my_bike.wheelie()        # Unique method of Bike
my_bike.display_info()   # Overridden method
my_bike.stop()           # Inherited method
```